

SafeSite AI

Maintained File Catalog

Edition	Complete final architecture
Updated	2026-09-01
Maintained files	143
Current milestone	Complete final architecture

This document explains the role, reason, and latest meaningful change for every maintained SafeSite AI project file. Generated caches, local secrets, Docker volumes, downloaded models, and temporary render files are intentionally excluded.

Maintenance protocol

Whenever a file is created, removed, or meaningfully changed, update the Markdown catalog and the generator metadata, regenerate this PDF, render every page, and include the documentation update in the same Git commit.

Root configuration

.gitattributes

Role: Marks generated PDFs, images, videos, and model weights as binary Git artifacts.

Why it exists: Prevents line-ending conversion, whitespace checks, and unreadable text diffs for binary files.

Latest change: Added binary rules for PDF, image, video, and PyTorch model artifacts.

.env.example

Role: Documents local configuration for PostgreSQL, FastAPI, MinIO, and streaming workers.

Why it exists: Lets developers create a local .env file without committing real credentials.

Latest change: Added read-only agent, Ollama text and vision models, dashboard, and final-stack endpoints.

.gitignore

Role: Excludes local, generated, cached, secret, and heavy files from Git.

Why it exists: Keeps commits focused on maintained source files.

Latest change: Excluded the generated verified tracking wheel alongside local binary dependencies.

README.md

Role: Provides the project overview, startup commands, learning links, and working rules.

Why it exists: Acts as the main entry point for every developer and reviewer.

Latest change: Added the final one-command architecture, service URLs, completion guide, and Lessons 30-32.

compose.yaml

Role: Defines the complete 18-service SafeSite architecture and its optional profiles.

Why it exists: Starts the architecture reproducibly and preserves data in named volumes.

Latest change: Added agent, dashboard, MLflow, monitoring, and Airflow profiles with explicit dependencies.

Database

database/init/001_schema.sql

Role: Creates the violations table, constraints, indexes, and fake seed events.

Why it exists: Establishes the structured event contract inside PostgreSQL.

Latest change: Initial camera, track, violation, confidence, evidence, and timestamp fields added.

database/init/002_event_idempotency.sql

Role: Adds the event key column, SHA256 format constraint, and unique partial index.

Why it exists: Makes retries and concurrent publication safe at the authoritative database layer.

Latest change: Added idempotent migration statements for fresh and existing PostgreSQL volumes.

database/init/003_agent_readonly.sql

Role: Creates the local SafeSite assistant role with read-only transactions and SELECT-only privileges.

Why it exists: Provides database-enforced protection even if an application-layer agent check fails.

Latest change: Added a repeatable local role, CONNECT and schema access, table SELECT, and default read-only mode.

API container

services/api/Dockerfile

Role: Builds the container image that runs the FastAPI application.

Why it exists: Packages Python, dependencies, source code, and the startup command reproducibly.

Latest change: Split the image into reusable base, runtime, and test build stages.

services/api/requirements.txt

Role: Declares the Python dependencies required by the API.

Why it exists: Makes dependency installation repeatable in the container.

Latest change: FastAPI, Uvicorn, Psycopg, pool, and settings dependencies added.

services/api/requirements-dev.txt

Role: Declares dependencies used only by automated API tests.

Why it exists: Keeps Pytest and HTTPX separate from the production runtime image.

Latest change: Added Pytest and HTTPX for black-box integration testing.

API application

services/api/app/__init__.py

Role: Marks the app directory as a Python package.

Why it exists: Makes imports such as app.main and app.schemas predictable.

Latest change: Initial package marker added.

services/api/app/config.py

Role: Loads the PostgreSQL connection URL from environment configuration.

Why it exists: Separates deployment configuration from application logic.

Latest change: Initial Pydantic settings model and development default added.

services/api/app/database.py

Role: Creates and manages the asynchronous PostgreSQL connection pool.

Why it exists: Reuses connections efficiently and manages them with the API lifecycle.

Latest change: Initial dictionary-row pool, startup wait, and shutdown functions added.

services/api/app/schemas.py

Role: Defines Pydantic request and response contracts for violations and statistics.

Why it exists: Validates JSON fields, types, ranges, and allowed violation values.

Latest change: Exposed the optional 64-character database event key in violation responses.

services/api/app/main.py

Role: Defines the FastAPI application lifecycle and HTTP routes.

Why it exists: Exposes health, create, list, filter, and summary operations over PostgreSQL.

Latest change: Added canonical SHA256 keys and conflict-safe 201-new/200-duplicate semantics.

API tests

services/api/tests/test_time_filters.py

Role: Tests time filtering, invalid ranges, and SQL-injection resistance against the running stack.

Why it exists: Proves that HTTP, validation, SQL, and PostgreSQL work correctly together.

Latest change: Added a fourth integration test proving duplicate POSTs create exactly one row.

Ingestion container

services/ingestion/Dockerfile

Role: Builds the reproducible CPU environment for video ingestion and YOLO inference.

Why it exists: Pins the official Ultralytics OpenCV, PyTorch, and YOLO runtime.

Latest change: Added offline installation of pinned Kafka, MinIO, and ByteTrack client wheels.

services/ingestion/requirements.txt

Role: Pins the Python clients and packages used for Kafka, MinIO, and Parquet.

Why it exists: Makes the producer image repeatable instead of resolving new client versions during each build.

Latest change: Added pinned PyArrow for typed, compressed Parquet tables.

Ingestion application

services/ingestion/app/__init__.py

Role: Marks the ingestion app directory as a Python package.

Why it exists: Allows tools to run consistently with `python -m app..`

Latest change: Initial package marker added.

services/ingestion/app/generate_demo_video.py

Role: Generates a controlled MP4 with known timing, resolution, and frame count.

Why it exists: Provides deterministic input without relying on personal or downloaded footage.

Latest change: Added configurable generation and a moving PPE-equipped worker scene.

services/ingestion/app/generate_violation_fixture.py

Role: Turns one held-out real image into a deterministic short MP4 with slow camera motion.

Why it exists: Provides repeatable temporal input without claiming real-motion evaluation.

Latest change: Added validation, protected output, metadata sidecar, and an evaluation warning.

services/ingestion/app/build_violation_review_reel.py

Role: Compiles labelled no-helmet construction images into a review MP4, first-frame proof, and JSON summary.

Why it exists: Gives the dashboard real labelled violation evidence while clearly separating it from continuous camera footage.

Latest change: Added deterministic video generation from the Construction-PPE no-helmet visual review set.

services/ingestion/app/sample_frames.py

Role: Samples video frames by time and writes JPEG evidence plus JSONL metadata.

Why it exists: Establishes the ingestion contract later consumed by Kafka, MinIO, and YOLO.

Latest change: Added validation, time-based sampling, traceable names, and metadata records.

services/ingestion/app/detect_image.py

Role: Runs YOLO on one image and writes annotated JPEG and structured JSON outputs.

Why it exists: Exposes classes, confidence scores, and boxes for later inference and event logic.

Latest change: Extracted shared detection serialization for identical single-image and batch box contracts.

services/ingestion/app/detect_frames.py

Role: Loads YOLO once and streams every image referenced by a JSONL frame manifest.

Why it exists: Preserves camera and timestamp context while writing scalable visual and structured outputs.

Latest change: Added manifest validation, batch inference, shared boxes, output protection, counts, and timing.

services/ingestion/app/track_frames.py

Role: Runs YOLO and ByteTrack over ordered manifest frames and writes temporary object identities.

Why it exists: Connects detections through time so violation logic can measure duration and avoid duplicate alerts.

Latest change: Added persistent tracking, visual and JSONL outputs, track coverage, spans, and timing.

services/ingestion/app/associate_ppe_tracks.py

Role: Associates supported PPE detections with stable person track IDs and exports evidence histories.

Why it exists: Connects tracking and PPE inference without rerunning models or treating missing boxes as violations.

Latest change: Added body-region matching, one-owner selection, duplicate suppression, visuals, and summaries.

services/ingestion/app/temporal_violations.py

Role: Converts direct per-track violation evidence into deduplicated temporal candidate events.

Why it exists: Separates noisy frame predictions from incidents with windows, opposite evidence, and cooldown.

Latest change: Extracted a reusable stateful temporal engine shared by offline and Kafka processing.

services/ingestion/app/publish_candidate_events.py

Role: Validates temporal candidates and optionally publishes supported events to FastAPI.

Why it exists: Adapts video candidates while protecting PostgreSQL with dry runs and type filtering.

Latest change: Preferred candidate occurred_at with historical base time as a compatibility fallback.

services/ingestion/app/publish_frames.py

Role: Uploads sampled JPEGs to MinIO and publishes their validated metadata to Kafka.

Why it exists: Creates the first asynchronous frame path while keeping large image bytes outside the broker.

Latest change: Added a validated starting sample index for resumable publication without earlier records.

services/ingestion/app/consume_frames.py

Role: Consumes validated frame events from Kafka and downloads their JPEG objects from MinIO.

Why it exists: Gives inference a replayable, offset-controlled input without coupling it to the producer.

Latest change: Added version-2 validation for timezone-aware camera captured_at timestamps.

services/ingestion/app/infer_frame_events.py

Role: Consumes raw frame events, runs YOLO, stores Silver outputs, and publishes detection events.

Why it exists: Turns asynchronous frame notifications into reproducible inference without blocking producers.

Latest change: Propagated captured_at, added heartbeats, and preserved bounded continuous processing.

services/ingestion/app/process_ppe_stream.py

Role: Consumes ordered PPE events, runs ByteTrack, associates PPE, evaluates temporal rules, and publishes outputs.

Why it exists: Connects Silver detections to worker evidence, MinIO Gold, track events, and confirmed candidates.

Latest change: Added isolated three-camera state, absolute time, heartbeats, and safe per-event commits.

services/ingestion/app/consume_candidate_events.py

Role: Consumes confirmed Kafka candidates and publishes supported violations through FastAPI.

Why it exists: Bridges asynchronous events to PostgreSQL with retries, duplicate handling, DLQ, and commit-last offsets.

Latest change: Added continuous delivery, occurred_at, heartbeats, and missing-time dead letters.

services/ingestion/app/stream_camera.py

Role: Samples camera, RTSP, or video sources and publishes timestamped Bronze frame events.

Why it exists: Provides a long-running producer with object evidence, Kafka keys, and absolute event time.

Latest change: Added version-2 events, JPEG encoding, looping, pacing, deterministic IDs, and heartbeats.

services/ingestion/app/worker_runtime.py

Role: Atomically writes throttled worker heartbeat records.

Why it exists: Provides one consistent liveness contract for every long-running service.

Latest change: Added worker status, processed counts, timestamps, and extensible details.

services/ingestion/app/monitor_streaming.py

Role: Monitors Kafka lag, heartbeats, FastAPI health, and MinIO liveness.

Why it exists: Detects stuck workers, growing queues, stale services, and unavailable dependencies.

Latest change: Added partition lag, alerts, atomic reports, continuous checks, and degraded exits.

services/ingestion/app/aggregate_gold.py

Role: Builds typed frame, worker-evidence, and daily-camera Parquet tables from Gold JSON.

Why it exists: Makes repeated analytics efficient while preserving event and evidence traceability.

Latest change: Added strict validation, Arrow schemas, Zstandard compression, atomic files, and summaries.

services/ingestion/app/validate_gold_quality.py

Role: Validates Gold Parquet schemas, identities, times, counts, relationships, and totals.

Why it exists: Stops duplicated or inconsistent analytics data from being accepted silently.

Latest change: Added 12 named checks, JSON reports, and a failing exit code.

services/ingestion/app/run_gold_analytics.py

Role: Runs Gold aggregation and its quality gate as one command.

Why it exists: Gives Compose one success boundary for both data creation and validation.

Latest change: Added combined reporting and quality failure propagation.

services/ingestion/app/render_yolo_labels.py

Role: Renders deterministic YOLO ground-truth samples as annotated images and contact sheets.

Why it exists: Exposes box quality, class semantics, and domain mismatch before training consumes labels.

Latest change: Added seeded selection, class filtering, pixel conversion, protected outputs, and JSON indexes.

services/ingestion/app/train_ppe_baseline.py

Role: Runs protected and reproducible YOLO PPE training and exports a SafeSite metrics summary.

Why it exists: Separates pipeline verification from model-quality claims while preserving exact settings.

Latest change: Added validation, absolute data config, deterministic arguments, protection, and JSON metrics.

Dashboard

services/dashboard/Dockerfile

Role: Packages the Streamlit dashboard as a reproducible container.

Why it exists: Lets the complete Compose stack run the UI without a host Python environment.

Latest change: Added the pinned dashboard runtime, source copy, health check, and port 8502 command.

services/dashboard/app.py

Role: Presents one professional real-video evidence frame with the no-vest decision, review confidence, precision, recall, and mAP50.

Why it exists: Gives a PFE reviewer a simple visual safety decision without technical tables or distracting video playback.

Latest change: Changed the final demo to a frame-only no-vest evidence screen backed by real Pexels footage.

services/dashboard/requirements.txt

Role: Declares the Streamlit dependency used by the host dashboard.

Why it exists: Keeps UI dependencies reproducible and separate from the API image.

Latest change: Added the supported Streamlit 1.x version range.

MLOps

services/mlops/Dockerfile

Role: Packages the local MLflow tracking and registry server.

Why it exists: Runs the same project database and artifact layout through Compose.

Latest change: Replaced the fragile local pip build with the official MLflow 3.15.2 image pinned by digest.

services/mlops/requirements.txt

Role: Pins the MLflow version used by the local tracking environment.

Why it exists: Keeps MLOps dependencies isolated from API and inference runtimes.

Latest change: Added MLflow 3.15.2 for Block 29.

services/mlops/log_ppe_experiment.py

Role: Logs PPE settings, metrics, evidence, and a packaged model into MLflow.

Why it exists: Connects one exact training result to reproducible artifacts and a registry version.

Latest change: Added hashes, 12 parameters, 54 metrics, nine artifacts, and candidate registration.

services/mlops/safesite_yolo_pyfunc.py

Role: Defines the model-from-code interface for packaged YOLO inference.

Why it exists: Makes model loading inspectable and avoids fragile live-object serialization.

Latest change: Added validated image inputs, lazy Ultralytics loading, and JSON detections.

services/mlops/compare_ppe_runs.py

Role: Logs or reuses PPE experiments and ranks completed MLflow runs by explicit metrics.

Why it exists: Makes candidate selection evidence-based while ignoring failed partial runs.

Latest change: Added safe metric names, mAP50-95 and recall ranking, and an atomic comparison report.

services/mlops/test_compare_ppe_runs.py

Role: Tests MLflow metric sanitization and deterministic run-ranking rules.

Why it exists: Prevents model selection from changing silently when metrics tie or contain unsafe names.

Latest change: Added three unit tests for ranking, recall tie-breaking, and MLflow-safe names.

services/mlops/README.md

Role: Documents the no-Docker MLflow workflow, stores, registry policy, and UI.

Why it exists: Lets developers reproduce and understand the local MLOps proof.

Latest change: Added two-run comparison commands and the experimental promotion boundary.

Grounded agent

services/agent/Dockerfile

Role: Packages the FastAPI and LangGraph assistant service.

Why it exists: Keeps agent dependencies and its port separate from the safety-event API.

Latest change: Added the pinned image, health check, data mount contract, and port 8010 runtime.

services/agent/requirements.txt

Role: Pins FastAPI, LangGraph, Psycpg, HTTPX, Pillow, and Uvicorn for the agent.

Why it exists: Makes text, database, and image interactions reproducible.

Latest change: Added exact versions verified by the local text, vision, and database proofs.

services/agent/run_windows.py

Role: Starts Uvicorn with a Windows selector event loop.

Why it exists: Avoids the Psycpg incompatibility with the default Proactor loop during local proofs.

Latest change: Added an explicit SelectorEventLoop runner and programmatic Uvicorn startup.

services/agent/README.md

Role: Documents the assistant flow, safety layers, endpoints, models, and proof commands.

Why it exists: Explains why the service is grounded and why the VLM cannot authorize violations.

Latest change: Added read-only database and Ollama text/vision instructions.

services/agent/app/__init__.py

Role: Marks the agent application directory as a Python package.

Why it exists: Supports stable module imports in local and container runtimes.

Latest change: Added the agent package marker.

services/agent/app/schemas.py

Role: Defines request, answer, query-plan, and vision response contracts.

Why it exists: Keeps LangGraph state and HTTP payloads explicit and validated.

Latest change: Added grounded-answer metadata including SQL, parameters, rows, and provider.

services/agent/app/query_catalog.py

Role: Maps supported natural-language intents to approved parameterized SELECT templates.

Why it exists: Prevents unrestricted model-generated SQL and blocks write vocabulary before database access.

Latest change: Added six deterministic analytical intents, exact allowlist validation, and table restrictions.

services/agent/app/database.py

Role: Manages the assistant PostgreSQL pool and read-only query transactions.

Why it exists: Enforces read-only execution at both connection-role and transaction levels.

Latest change: Added health evidence for the current role and transaction_read_only setting.

services/agent/app/ollama.py

Role: Calls local Ollama chat and vision endpoints.

Why it exists: Separates model serving from agent orchestration and supports deterministic fallback behavior.

Latest change: Added model health, grounded row summaries, base64 images, and uncertainty-aware PPE prompts.

services/agent/app/graph.py

Role: Defines the LangGraph plan, validate, execute, and answer workflow.

Why it exists: Makes every agent decision stage explicit, testable, and observable.

Latest change: Added approved-query execution and deterministic answers when Ollama is unavailable.

services/agent/app/main.py

Role: Exposes health, grounded question, and evidence-description HTTP endpoints.

Why it exists: Provides one safe API around LangGraph, PostgreSQL, Ollama, and local evidence files.

Latest change: Added 422 rejection, verified JPEG/PNG path confinement, and supporting-context warnings.

services/agent/tests/test_agent.py

Role: Tests approved queries, rejections, fallback answers, HTTP contracts, and image-path safety.

Why it exists: Proves unsafe requests stop before database execution and every catalog plan validates.

Latest change: Added seven passing agent tests.

Monitoring and data quality

services/monitoring/Dockerfile

Role: Packages drift and Great Expectations jobs.

Why it exists: Keeps monitoring dependencies separate and runnable through Compose profiles.

Latest change: Added the pinned Python monitoring runtime and mounted-project execution contract.

services/monitoring/requirements.txt

Role: Pins NumPy, Pillow, and Great Expectations.

Why it exists: Makes image-distribution and event-quality checks reproducible.

Latest change: Added versions verified by the four monitoring tests and real reports.

services/monitoring/analyze_drift.py

Role: Compares image brightness, sharpness, and model-confidence distributions with PSI.

Why it exists: Provides an early warning when current data differs from the reference data.

Latest change: Added stable, warning, and drift thresholds plus atomic JSON reporting.

services/monitoring/validate_events.py

Role: Runs Great Expectations against exported violation events.

Why it exists: Stops downstream work when required fields, types, ranges, or uniqueness fail.

Latest change: Added 21 expectations and failure-propagating atomic reports.

services/monitoring/test_analyze_drift.py

Role: Tests stable and strongly shifted drift comparisons.

Why it exists: Protects PSI behavior and classification thresholds from regression.

Latest change: Added two deterministic drift tests.

services/monitoring/test_validate_events.py

Role: Tests valid and invalid Great Expectations event batches.

Why it exists: Proves bad confidence and unsupported violation types fail quality validation.

Latest change: Added two deterministic event-quality tests.

services/monitoring/README.md

Role: Documents drift interpretation, quality checks, commands, and limits.

Why it exists: Prevents a drift signal from being misrepresented as measured model inaccuracy.

Latest change: Added the verified PSI and 21-expectation workflows.

Learning documentation

docs/DATA_SOURCES.md

Role: Records provenance, licensing, retrieval details, measured metadata, and appropriate use for external data.

Why it exists: Makes experiments legally traceable and reproducible without committing downloaded media.

Latest change: Added the SH17 PPE dataset candidate review, including its license boundary and missing-helmet inference limitation.

docs/REAL_DATASET_UPGRADE_PLAN.md

Role: Explains how to move from the current MVP proof to stronger real-data violation detection.

Why it exists: Separates completed architecture work from remaining ML evidence work so the project story stays technically honest.

Latest change: Added the SH17 candidate assessment, dashboard proof status, model-readiness gap, and next upgrade steps.

docs/MODEL_SOURCES.md

Role: Records model provenance, runtime version, artifact hash, class limits, and license boundaries.

Why it exists: Makes the baseline reproducible and prevents unknown weights from entering experiments.

Latest change: Added pinned lap wheel provenance, SHA256, platform, and offline installation rationale.

docs/ARCHITECTURE.md

Role: Explains the architecture by following one PPE violation event.

Why it exists: Builds a mental model connecting every planned system layer.

Latest change: Added the final agent, VLM, quality, drift, Airflow, and architecture completion flow.

docs/FINAL_ARCHITECTURE.md

Role: Maps every final diagram layer to its source, runtime profile, proof, and trust boundary.

Why it exists: Provides the authoritative completion and demo reference for the full project.

Latest change: Updated the dashboard proof to describe its violation-first decision and human-review workflow.

docs/LESSON_01.md

Role: Contains the first guided lesson and practical exercises.

Why it exists: Teaches Docker, networking, validation, persistence, and event flow.

Latest change: Initial Day 1 exercises and explanation checkpoint added.

docs/LESSON_02.md

Role: Contains the second guided lesson and implementation exercise.

Why it exists: Teaches HTTP, async waiting, pooling, safe SQL, time filters, and tests.

Latest change: Initial Day 2 lesson and explanation checkpoint added.

docs/LESSON_03.md

Role: Contains the video-fundamentals lesson and frame-sampling exercise.

Why it exists: Teaches FPS, resolution, codecs, timestamps, sampling, and JSONL metadata.

Latest change: Added the Day 3 lesson, Docker commands, contract, and checkpoint.

docs/LESSON_04.md

Role: Documents real-footage provenance, visual assessment, limitations, and data leakage.

Why it exists: Teaches the difference between a smoke test and valid model evaluation.

Latest change: Added the Pexels workflow, 50-frame result, YOLO checklist, and video-level split rule.

docs/LESSON_05.md

Role: Documents the first real YOLO inference and its prerequisite detection concepts.

Why it exists: Connects boxes, confidence, IoU, precision, recall, and mAP to actual outputs.

Latest change: Added the 50-frame workflow, confidence and throughput results, box scaling, and accuracy warning.

docs/LESSON_06.md

Role: Documents multi-object tracking concepts and the first ByteTrack continuity experiment.

Why it exists: Teaches temporary IDs, association, lifecycle, event deduplication, and stability limits.

Latest change: Added the conceptual lesson, command, output contracts, and honest interpretation guidance.

docs/LESSON_07.md

Role: Documents PPE dataset selection, acquisition, structural and visual audits, balance, and limitations.

Why it exists: Teaches why verified structure and sampled boxes still require domain-specific validation.

Latest change: Added reproducible visual review, measured samples, domain mismatch, and baseline-only decision.

docs/LESSON_08.md

Role: Documents the first PPE training smoke test, settings, outputs, metrics, and interpretation.

Why it exists: Teaches why pipeline success and saved weights do not prove that a model is accurate.

Latest change: Added the CPU command, 113-image run, full validation, outputs, and quality conclusion.

docs/LESSON_09.md

Role: Documents the ten-epoch PPE baseline, trends, class metrics, visuals, and readiness decision.

Why it exists: Teaches how losses, mAP, recall, confusion matrices, and predictions support one conclusion.

Latest change: Linked the baseline conclusion to the completed real-video evaluation in Lesson 10.

docs/LESSON_10.md

Role: Documents trained PPE inference on 50 unseen construction frames at two thresholds.

Why it exists: Teaches generalization, flicker, duplicates, threshold trade-offs, and temporal event rules.

Latest change: Added frame coverage, duplicate counts, compliant-worker interpretation, and safe next steps.

docs/LESSON_11.md

Role: Documents PPE-to-worker association and per-track evidence across 50 frames.

Why it exists: Teaches stage separation, spatial matching, duplicate suppression, and unknown versus violation.

Latest change: Added the reproducible experiment, track measurements, architecture, limits, and safety rule.

docs/LESSON_12.md

Role: Documents temporal rules that convert repeated direct evidence into one candidate incident.

Why it exists: Teaches windows, thresholds, opposite evidence, cooldown, independent keys, and contract limits.

Latest change: Added synthetic proofs, zero-event real-video result, outputs, API mismatch, and tuning limits.

docs/LESSON_13.md

Role: Documents the safe adapter from candidate JSONL to FastAPI and PostgreSQL.

Why it exists: Teaches contract mapping, absolute time, dry runs, filtering, and database idempotency.

Latest change: Replaced race-prone pre-checks with event keys and PostgreSQL uniqueness.

docs/LESSON_14.md

Role: Explains the Kafka broker and MinIO object-storage infrastructure foundation.

Why it exists: Teaches topics, partitions, offsets, lag, object tiers, listeners, KRaft, and persistence.

Latest change: Added the topic restart proof, MinIO health checks, addresses, and production limits.

docs/LESSON_15.md

Role: Explains the first real sampled-frame publication through MinIO and Kafka.

Why it exists: Teaches object URIs, Kafka keys, upload ordering, delivery confirmation, and replay boundaries.

Latest change: Added the verified event contract, independent storage and broker proof, and consumer design.

docs/LESSON_16.md

Role: Explains the first Kafka consumer and its verified MinIO frame download.

Why it exists: Teaches groups, offsets, lag, manual commits, at-least-once processing, and atomic files.

Latest change: Added the offset proof, zero lag, matching SHA256, reproduction command, and YOLO handoff.

docs/LESSON_17.md

Role: Explains the first complete Kafka, MinIO, and YOLO streaming inference event.

Why it exists: Teaches Bronze-to-Silver processing, deterministic identity, model reuse, topics, and commits.

Latest change: Added the 11-person smoke test, Silver objects, event readback, limits, and reproduction command.

docs/LESSON_18.md

Role: Explains the complete 30-frame stream through the trained experimental PPE model.

Why it exists: Teaches resumable publication, partition order, full-stream proof, classes, and missing evidence.

Latest change: Added 30-event and 60-object proof, 145-box results, zero lag, limits, and tracking handoff.

docs/LESSON_19.md

Role: Explains streamed tracking, PPE association, temporal evaluation, and Gold evidence.

Why it exists: Teaches stateful processing, track continuity, unknown evidence, commits, and measured outputs.

Latest change: Added the 30-event run, 14 IDs, 80 associations, 60 Gold objects, and zero candidates.

docs/LESSON_20.md

Role: Explains how confirmed Kafka candidates reach FastAPI and PostgreSQL safely.

Why it exists: Teaches HTTP adaptation, absolute time, retries, idempotency, DLQ, and offset boundaries.

Latest change: Added new-event, replay, and unsupported-type proofs with one row and zero source lag.

docs/LESSON_21.md

Role: Explains the complete isolated streaming run and its final checkpoint report.

Why it exists: Teaches orchestration, topic isolation, candidate approval, verification, and failure boundaries.

Latest change: Added the ten-stage command, 30/30/30 Kafka proof, 30/60/60 storage proof, and guards.

docs/LESSON_22.md

Role: Explains monitoring saved streaming runs beside API-backed violation events.

Why it exists: Teaches pipeline completeness, model accuracy, candidate approval, and persistence as separate ideas.

Latest change: Added run selection, stage and Kafka checkpoints, CV metrics, safety meaning, and limits.

docs/LESSON_23.md

Role: Explains independent continuous PPE inference and tracking worker services.

Why it exists: Teaches service lifecycle, waiting, isolation, bounded memory, restart behavior, and commit safety.

Latest change: Added streaming services, 35-second idle proof, one-frame regression, and the delivery boundary.

docs/LESSON_24.md

Role: Explains continuous camera ingestion and absolute event time.

Why it exists: Teaches Bronze publication, version 2, captured_at, monotonic time, and loop-safe simulation.

Latest change: Added the three-camera timestamp proof and hardware-time limitation.

docs/LESSON_25.md

Role: Explains multi-camera Kafka partitioning and isolated tracking state.

Why it exists: Teaches key order, shared partitions, scaling, and camera-scoped identities.

Latest change: Added the three-camera, six-identity, zero-collision proof.

docs/LESSON_26.md

Role: Explains always-on candidate delivery with correct absolute event time.

Why it exists: Teaches timestamp propagation, retries, idempotency, commits, and missing-time DLQ safety.

Latest change: Added exact PostgreSQL time and missing-time dead-letter proofs.

docs/LESSON_27.md

Role: Explains heartbeats, Kafka lag, dependency checks, and alerts.

Why it exists: Teaches why process liveness alone cannot prove pipeline health.

Latest change: Added five-service health, zero-restart, 1/1/0 lag, and stale-alert proofs.

docs/LESSON_28.md

Role: Explains Gold Parquet aggregation, data quality, and daily Airflow orchestration.

Why it exists: Teaches evidence versus analytics, quality gates, and workflow scheduling.

Latest change: Added the three-table design, 12-check proof, duplicate rejection, and commands.

docs/LESSON_29.md

Role: Explains MLflow runs, stores, model versions, aliases, and promotion boundaries.

Why it exists: Teaches model traceability using the verified SafeSite baseline.

Latest change: Added the local SQLite proof, packaging workflow, counts, and honest limitations.

docs/LESSON_30.md

Role: Explains Great Expectations event validation and PSI-based distribution drift.

Why it exists: Teaches the difference between structural data quality, input change, and measured accuracy.

Latest change: Added the real 21-check quality result and three-distribution drift interpretation.

docs/LESSON_31.md

Role: Explains the read-only LangGraph agent and Ollama text/vision support.

Why it exists: Teaches grounding, SQL allowlists, database permissions, fallback answers, and VLM limits.

Latest change: Added real SELECT/DELETE, HTTP 422, Llama answer, and Qwen frame-description proofs.

docs/LESSON_32.md

Role: Walks one event through the complete architecture in simple language.

Why it exists: Provides a final mental model and a five-minute interview explanation structure.

Latest change: Added the 13-step flow, authority boundaries, completion evidence, and honest limitations.

docs/MVP_DEMO.md

Role: Provides the one-command demo, checkpoints, results, limits, and interview explanation.

Why it exists: Turns the local components into a reproducible and honest portfolio demonstration.

Latest change: Added the validated 30-frame run, duplicate replay, dashboard, and quality warning.

docs/ROADMAP.md

Role: Defines the realistic 28-day learning and implementation plan.

Why it exists: Protects the deadline using must-have, should-have, and stretch priorities.

Latest change: Initial plan adapted for a GTX 1650 with 4 GB VRAM.

docs/FILE_CATALOG.md

Role: Provides the human-readable source catalog for maintained project files.

Why it exists: Keeps file responsibilities and meaningful changes understandable.

Latest change: Added MLflow services, the UI launcher, Lesson 29, and Block 29 proof.

Orchestration

orchestration/airflow/Dockerfile

Role: Builds the Airflow runtime used by the daily Gold workflow.

Why it exists: Pins the scheduler image and adds the Parquet dependency required by SafeSite tasks.

Latest change: Added the Airflow 3.1 Python 3.12 base and requirements installation.

orchestration/airflow/requirements.txt

Role: Pins the PyArrow dependency used inside Airflow.

Why it exists: Allows scheduled Gold tasks to read and write Parquet consistently.

Latest change: Added PyArrow 21.0.0.

orchestration/airflow/dags/safesite_gold_daily.py

Role: Defines daily Gold aggregation followed by quality validation in Airflow.

Why it exists: Schedules analytics in the correct order and stops after task failure.

Latest change: Added a no-catchup, single-active-run DAG with compatible Bash imports.

orchestration/airflow/README.md

Role: Documents the environment, paths, and data mount required by the Gold DAG.

Why it exists: Separates the deployment contract from the lighter local execution proof.

Latest change: Added task flow, runtime requirements, and the local Airflow boundary.

Data acquisition

data/videos/external/pexels-32244795.mp4

Role: Stores the validated real Pexels construction-worker source video used for final evidence.

Why it exists: Provides real footage for the no-vest dashboard proof instead of generated or image-built media.

Latest change: Downloaded the free Pexels scaffolding inspection video validated by the user.

data/final-demo/real-pexels-32244795/proof-no-vest.jpg

Role: Shows the professional frame-only no-vest evidence used by the final dashboard.

Why it exists: Makes the real construction violation visually understandable without requiring video playback.

Latest change: Added a highlighted 2.0-second frame showing the worker with a helmet but no safety vest.

data/final-demo/summary.json

Role: Stores the real Pexels source, proof frame path, no-vest decision, and vest-class model metrics.

Why it exists: Lets the dashboard render the professional evidence screen without hard-coded values.

Latest change: Changed the final demo metadata from no-helmet image-built video to real no-vest evidence.

data/evaluation/no-helmet-construction-proof/no-helmet-ground-truth-review.mp4

Role: Shows a curated labelled PPE missing-helmet example as the dashboard's primary violation proof reel.

Why it exists: Gives the interface real ground-truth violation evidence while making clear it is not continuous camera footage.

Latest change: Added an 8-second MP4 from image1162 with 8 labelled no_helmet boxes.

data/evaluation/no-helmet-construction-proof/summary.json

Role: Records the curated proof reel source, selected image ID, label count, output paths, and limitation.

Why it exists: Lets the dashboard load proof metadata without hard-coding sample counts or file names.

Latest change: Added traceable metadata for the dashboard's primary no-helmet proof.

data/evaluation/no-helmet-construction-proof/first-frame.jpg

Role: Provides a still fallback for the curated no-helmet proof reel.

Why it exists: Keeps visual proof visible even when video playback is unavailable.

Latest change: Added the first rendered frame from the curated proof video.

data/evaluation/no-helmet-review/no-helmet-ground-truth-review.mp4

Role: Shows labelled real no-helmet dataset examples as a broader review reel.

Why it exists: Gives the dashboard visible violation proof without relying on fake database records.

Latest change: Added a 24-second MP4 from 12 labelled Construction-PPE images with 21 no_helmet boxes.

data/evaluation/no-helmet-review/summary.json

Role: Records the review reel source, sample count, labels, output paths, and limitation.

Why it exists: Makes the proof traceable and prevents confusion with continuous camera footage.

Latest change: Added the generated no-helmet review metadata used by the dashboard.

data/evaluation/no-helmet-review/first-frame.jpg

Role: Provides a still visual fallback from the no-helmet review reel.

Why it exists: Keeps proof visible when browser video playback fails or is slow.

Latest change: Added the first rendered ground-truth review frame.

data/videos/synthetic/mixed-ppe-six-workers-source.png

Role: Provides a controlled source scene with exactly six workers and three visible PPE violations.

Why it exists: Gives demonstrations obvious ground truth without presenting generated media as real footage.

Latest change: Added three compliant workers, two without helmets, and one without a vest.

data/videos/synthetic/mixed-ppe-six-workers-10s.mp4

Role: Animates the controlled scene as a ten-second, 10 FPS video for full-pipeline tests.

Why it exists: Exercises sampling, Kafka, MinIO, inference, tracking, and temporal decisions with known violations.

Latest change: Added the controlled video; six tracks were maintained but zero candidates were confirmed.

data/videos/synthetic/mixed-ppe-six-workers-10s.json

Role: Records synthetic provenance, video properties, worker count, and ground-truth violations.

Why it exists: Prevents the fixture from being mistaken for real footage or an accuracy benchmark.

Latest change: Added counts for three compliant workers, two NO_HELMET cases, and one NO_VEST case.

scripts/download_sample_video.py

Role: Downloads the licensed Pexels sample video and verifies its SHA256 hash.

Why it exists: Reproduces the exact external input without storing the media itself in Git.

Latest change: Added idempotent download, hash verification, temporary output, and provenance reporting.

scripts/download_construction_ppe_dataset.py

Role: Downloads, fingerprints, and safely extracts the official Construction-PPE archive.

Why it exists: Makes PPE training input reproducible while rejecting corrupt files and unsafe ZIP paths.

Latest change: Added trusted retrieval, size and hash checks, traversal protection, and idempotence.

scripts/audit_yolo_dataset.py

Role: Audits YOLO pairing, normalized boxes, class balance, and exact split duplicates.

Why it exists: Catches structural defects before training and writes machine-readable audit evidence.

Latest change: Added dependency-free config parsing, statistics, orphan checks, and SHA256 comparison.

scripts/download_yolo_model.py

Role: Downloads the pinned YOLO26n baseline and verifies its SHA256 hash.

Why it exists: Reproduces the exact model without committing or trusting an unverified binary.

Latest change: Added idempotent download, temporary output, checksum validation, and provenance reporting.

scripts/download_tracking_dependency.py

Role: Downloads the pinned Linux lap wheel and verifies its SHA256 before image builds.

Why it exists: Supplies ByteTrack reproducibly without disabling TLS verification inside Docker.

Latest change: Added Windows trust-store download, cleanup, platform pinning, and checksum validation.

scripts/download_streaming_dependencies.py

Role: Downloads Linux wheels for pinned Kafka, MinIO, and Parquet packages with SHA256 verification.

Why it exists: Supports reproducible offline Docker installation without disabling TLS inside the container.

Latest change: Added PyArrow to the exact 11-file checksum allowlist.

scripts/build_final_demo_video.py

Role: Builds the final frame-only no-vest evidence image and summary from the validated real Pexels video.

Why it exists: Keeps the final presentation proof reproducible instead of manually edited.

Latest change: Replaced the image-built demo generator with real-video evidence-frame extraction and annotation.

scripts/run_mvp_demo.ps1

Role: Runs every local MVP stage and optionally launches Streamlit with one command.

Why it exists: Provides a reproducible entry point while preserving intermediate artifacts.

Latest change: Replaced Path.GetRelativePath with Windows PowerShell-compatible path normalization.

scripts/run_streaming_demo.ps1

Role: Runs one isolated Kafka, MinIO, CV, API, and PostgreSQL demonstration.

Why it exists: Replaces manual sequencing with validated inputs, isolated topics, safety gates, and one report.

Latest change: Replaced Path.GetRelativePath with Windows PowerShell-compatible path normalization.

scripts/start_mlflow_ui.ps1

Role: Starts local MLflow against the project SQLite database and artifact directory.

Why it exists: Provides one Windows command that requires no Docker and supports spaced paths.

Latest change: Added a configurable localhost port and local-environment validation.

scripts/prove_agent_locally.ps1

Role: Runs an isolated PostgreSQL and agent proof without Docker.

Why it exists: Verifies the real read-only role, write rejection, HTTP grounding, and cleanup.

Latest change: Added timestamped clusters, migrations, Llama call, 422 check, and atomic proof JSON.

scripts/run_final_architecture.ps1

Role: Starts the complete SafeSite profiles and waits for their public HTTP endpoints.

Why it exists: Provides the requested one-command final architecture entry point.

Latest change: Creates all five three-partition Kafka topics before starting consumers, then launches every architecture profile.

scripts/verify_final_architecture.py

Role: Audits saved evidence for every final architecture layer.

Why it exists: Creates one machine-readable completion report instead of relying on undocumented claims.

Latest change: Verifies all 13 live services and the dashboard's decision, evidence, missing-proof, technical, and assistant sections.

Documentation tooling and output

scripts/generate_file_catalog_pdf.py

Role: Generates this polished catalog PDF from maintained metadata.

Why it exists: Makes future PDF updates consistent and repeatable.

Latest change: Recorded the reliable first-run MLflow and Kafka startup changes and kept the register on a clean page.

output/pdf/SafeSite_AI_File_Catalog.pdf

Role: Provides the printable and shareable form of the file catalog.

Why it exists: Supports project reviews, learning, and PFE interview preparation.

Latest change: Regenerated after making MLflow and Kafka startup reproducible.

Change register

2026-09-06: Added a controlled ten-second video with six workers and three visible ground-truth violations. The complete 30-frame run maintained six tracks but confirmed zero candidates, documenting a current model limitation rather than claiming compliance.

2026-09-03: Replaced the unreliable local MLflow dependency build with the official MLflow 3.15.2 image pinned by digest, made the launcher create all five three-partition Kafka topics before consumers start, added truthful full-Compose replay recording with live verification of all 13 long-running services, fixed Docker-safe dashboard data-root resolution, reorganized the dashboard around a clear safety decision with honest visual-proof status, and updated the final audit to enforce the new interface contract.

2026-09-01: Completed the 18-service architecture with the read-only LangGraph agent, Ollama Llama and Qwen-VL, Great Expectations, drift monitoring, two-run MLflow comparison, Airflow container, final launcher, 12-check completion audit, Lessons 30-32, and the final architecture matrix.

2026-08-27: Logged the existing PPE baseline with 12 parameters, 54 metrics, and nine run artifacts; packaged its exact weight with model-from-code; registered SafeSite-PPE-Detector version 1 as the experimental candidate; verified SQLite integrity and the local MLflow UI; and added Lesson 29.

2026-08-27: Added Gold frame, worker, and daily-camera Parquet tables, a 12-check quality gate, duplicate-event rejection, one-command analytics execution, the daily Airflow DAG contract, and Lesson 28.

2026-08-21: Added worker heartbeats, Kafka lag, FastAPI and MinIO checks, alerts, stale-report protection, and a five-service healthy proof with zero restarts, lag 1/1/0, and a degraded stopped-worker proof in Lesson 27.

2026-08-21: Added always-on candidate delivery using propagated occurred_at, exact PostgreSQL time preservation, retry and idempotency boundaries, and missing-time DLQ safety in Lesson 26.

2026-08-21: Added camera-key partitioning, isolated per-camera YOLO and temporal state, camera-scoped track identities, and a three-camera zero-collision proof in Lesson 25.

2026-08-21: Added continuous camera, RTSP, and video ingestion with MinIO Bronze, Kafka schema-version-2 events, absolute captured_at, monotonic stream time, and a three-camera proof in Lesson 24.

2026-08-21: Added configurable continuous PPE inference and tracking services, bounded worker memory, restartable output paths, per-event commit-after-output safety, a 35-second idle proof with zero restarts, and a one-frame regression in Lesson 23.

2026-08-21: Added Streamlit monitoring for selectable run reports, stage health, Kafka frame counts, CV measurements, candidate and database outcomes, technical JSON inspection, and honest safety interpretation in Lesson 22.

2026-08-21: Added and ran one isolated ten-stage streaming command; verified 30 raw, 30 PPE, and 30 track events, 30 Bronze, 60 Silver, and 60 Gold objects, zero lag, healthy API and database, zero unconfirmed writes, opt-in candidate delivery, and protected reruns in Lesson 21.

2026-08-20: Connected confirmed Kafka candidates to FastAPI and PostgreSQL with retries, HTTP 201/200 idempotency, commit-last offsets, and a confirmed dead-letter topic; proved one new row, one safe replay, one unsupported event in the DLQ, and zero source lag in Lesson 20.

2026-08-20: Processed 30 ordered PPE events through persistent ByteTrack, worker-level PPE association, sliding-window rules, MinIO Gold, and Kafka outputs; verified 30 unique track events, 60 non-empty Gold objects, zero lag, and zero unsupported safety accusations in Lesson 19.